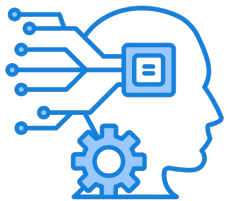
Universidade Federal de Goiás
Instituto de Informática
Prof. Dr. Ronaldo Martins da Costa

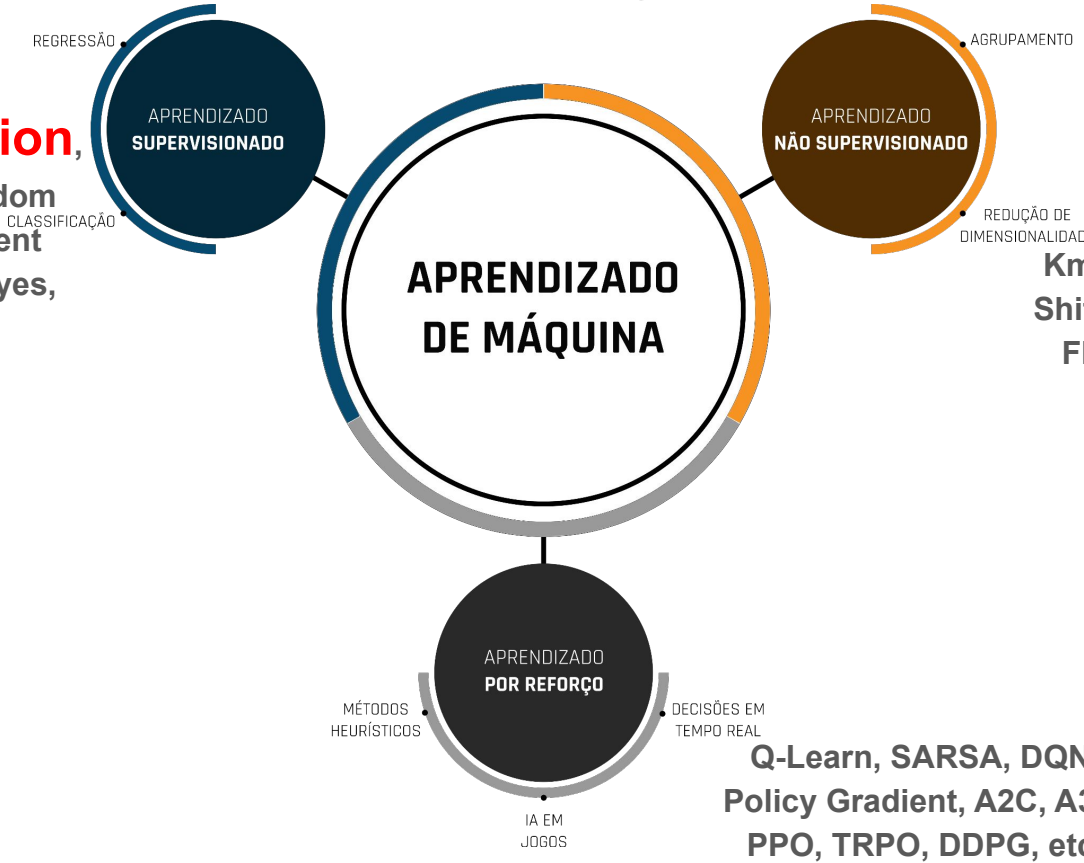
UFG
INF
INSTITUTO DE INFORMÁTICA



Aprendizado de Máquina - Paradigmas

Logistic Regression,

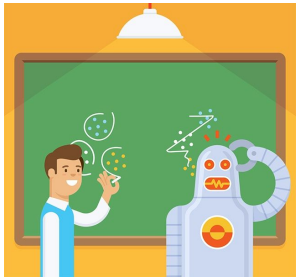
KNN, Decision Tree, Random Forest, Adaboost, Gradient Boosting, SVM, Naive Bayes, MLP, Deep Learning, Transformers, etc.



Kmeans, Dbscan, Mean Shift, PCA, t-SNE, Umap, FP-Growth, LDA, etc.

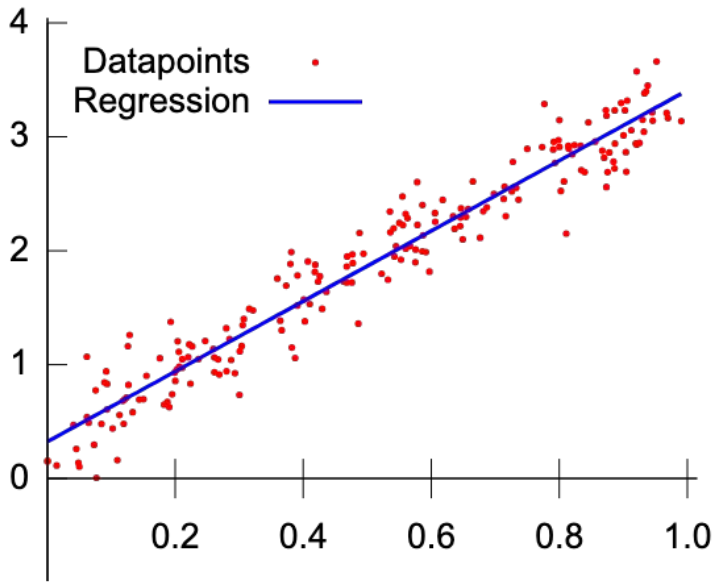
Q-Learn, SARSA, DQN, Policy Gradient, A2C, A3C, PPO, TRPO, DDPG, etc.

Aprendizado Supervisionado - Regressão



UFG INF
INSTITUTO DE INFORMÁTICA

➔ Utilizada para prever valores numéricos contínuos a partir da relação entre variáveis de entrada e uma variável de saída



Configurando o ambiente

➔ Construindo o ambiente

```
# Criando o ambiente virtual am-env entrando na pasta e ativando o ambiente:

python3 -m venv am-env

cd am-env
source bin/activate

# Criando uma pasta no ambiente virtual am-env para scripts python:

mkdir python_scripts
cd python_scripts
```

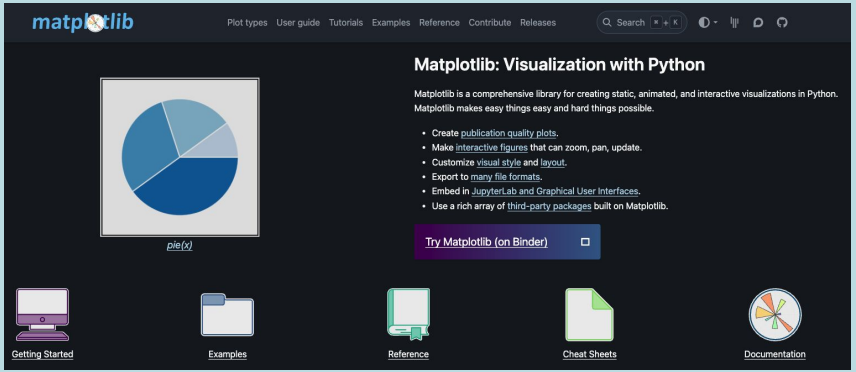
Configurando o ambiente

➔ Para os exemplos utilizaremos um ambiente Web com Django e biblioteca Plotly e um ambiente de scripts de python com a biblioteca MatPlotLib

Instalando os pacotes e configurando o ambiente

```
pip install matplotlib
pip install django
pip install plotly
django-admin startproject python_django

cd python_django
python manage.py startapp am
```



Configurando o ambiente

➔ Ajuste o arquivo settings.py para configurando o caminho da app **am**

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'am',
]
```

Configurando o ambiente

➔ /Users/ronaldomartinsdacosta/am-env/python_django/python_django/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('am.urls')),
    path('am/', include('am.urls')),
    ]
```

Configurando o ambiente

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/urls.py

```
from django.contrib import admin
from django.urls import path
from . import views

app_name = 'app_am'

urlpatterns = [
    path('', views.am_exemplo_01, name='am_exemplo_01'),
    path('am_exemplo_01', views.am_exemplo_01, name='am_exemplo_01'),
]
```

Configurando o ambiente

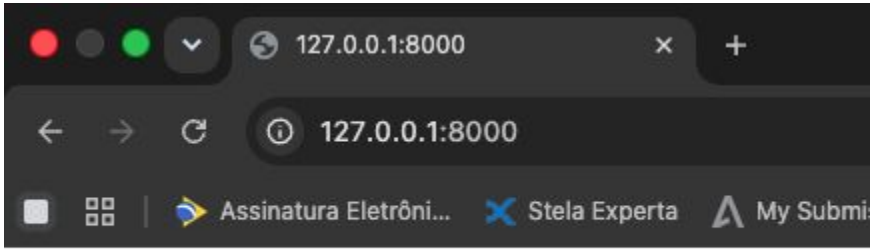
➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/views.py

```
from django.shortcuts import render
from django.http import HttpResponse

def am_exemplo_01(request):
    return HttpResponse("Exemplos de Aprendizado de Máquina (AM)")
```

Configurando o ambiente

```
python manage.py runserver
```



Exemplos de Aprendizado de Máquina (AM)



Revisando Python - Listas - listas.py

```
# Criando uma lista com 5 nomes
nomes = ["Ana", "Bruno", "Carlos", "Daniela", "Eduardo"]

print("Lista inicial:", nomes)

# Inserir um nome na posição 3 (índice 2)
nomes.insert(2, "Fernanda")

# Inserir um nome no final da lista
nomes.append("Gabriel")

# Inserir um nome no início da lista
nomes.insert(0, "Helena")

# Remover o 2° elemento da lista (índice 1)
nomes.pop(1)

print("Lista final:", nomes)
```



Revisando Python - Tuplas - tuplas.py

```
# Criando uma tupla com 5 pessoas
pessoas = (
    ("Ana", 1995),
    ("Bruno", 1990),
    ("Carlos", 1988),
    ("Daniela", 1997),
    ("Eduardo", 1992)
)

print("Tupla inicial:")
print(pessoas)

# Inserir uma pessoa na posição 3
pessoas = pessoas[:2] + (("Fernanda", 1993),) + pessoas[2:]

# Inserir uma pessoa no final
pessoas = pessoas + (("Gabriel", 1998),)
```



Revisando Python - Tuplas - tuplas.py

```
# Inserir uma pessoa no início
pessoas = (("Helena", 1991),) + pessoas

# Remover o 2° elemento (índice 1)
pessoas = pessoas[:1] + pessoas[2:]

print("\nTupla final:")
print(pessoas)
```



Revisando Python - Tuplas - tuplas.py

```
cliente = ("Ana", 1995)

# Sem unpacking
nome = cliente[0]
ano = cliente[1]

print(nome)
print(ano)

# Com unpacking
nome, ano = cliente

print(nome)
print(ano)
```



Revisando Python - Tuplas

- Por que Tuplas ?
  - Tuplas são imutáveis e normalmente contém elementos de tipos diferentes que são acessados via unpacking ou indexação
  - Tuplas geralmente são úteis para unir um conjunto pequeno de dados em uma única variável, e indexa-lo ou desempacota-lo quando necessário
  - Passagem de parâmetros em funções – A função pode ter uma única variável como entrada, porém esta pode ser uma tupla, o que permite uma passagem de parâmetros bastante flexível



Revisando Python - Dicionários - dicionarios.py

```
# Criando um dicionário
pessoa = {
    "nome": "Ana",
    "ano_nascimento": 1995,
    "cidade": "Goiânia"
}

print("Dicionário inicial:")
print(pessoa)

# Acessando valores
print("\nNome:", pessoa["nome"])
print("Cidade:", pessoa["cidade"])

# Adicionando um novo elemento
pessoa["profissao"] = "Engenheira"
```



Revisando Python - Dicionários - dicionarios.py

```
# Alterando um valor existente
pessoa["cidade"] = "Brasília"

print("\nApós inclusão e alteração:")
print(pessoa)

# Verificando se uma chave existe
if "nome" in pessoa:
    print("\nA chave 'nome' existe.")

# Obtendo um valor com get()
print("Telefone:", pessoa.get("telefone", "Não informado"))

# Listando chaves
print("\nChaves:")
for chave in pessoa.keys():
    print(chave)
```



Revisando Python - Dicionários - dicionarios.py

```
# Listando valores
print("\nValores:")
for valor in pessoa.values():
    print(valor)

# Listando chave e valor
print("\nItens:")
for chave, valor in pessoa.items():
    print(f"{chave}: {valor}")

# Removendo um elemento específico
del pessoa["profissao"]

# Outra forma de remover
cidade = pessoa.pop("cidade")

print("\nCidade removida:", cidade)
```



Revisando Python - Dicionários - dicionarios.py

```
print("\nApós remoções:")
print(pessoa)

# Quantidade de elementos
print("\nQuantidade de elementos:", len(pessoa))

# Copiando o dicionário
copia = pessoa.copy()

# Limpando o dicionário
pessoa.clear()

print("\nDicionário original após clear():")
print(pessoa)

print("\nCópia preservada:")
print(copia)
```



Numerical Python - NumPy



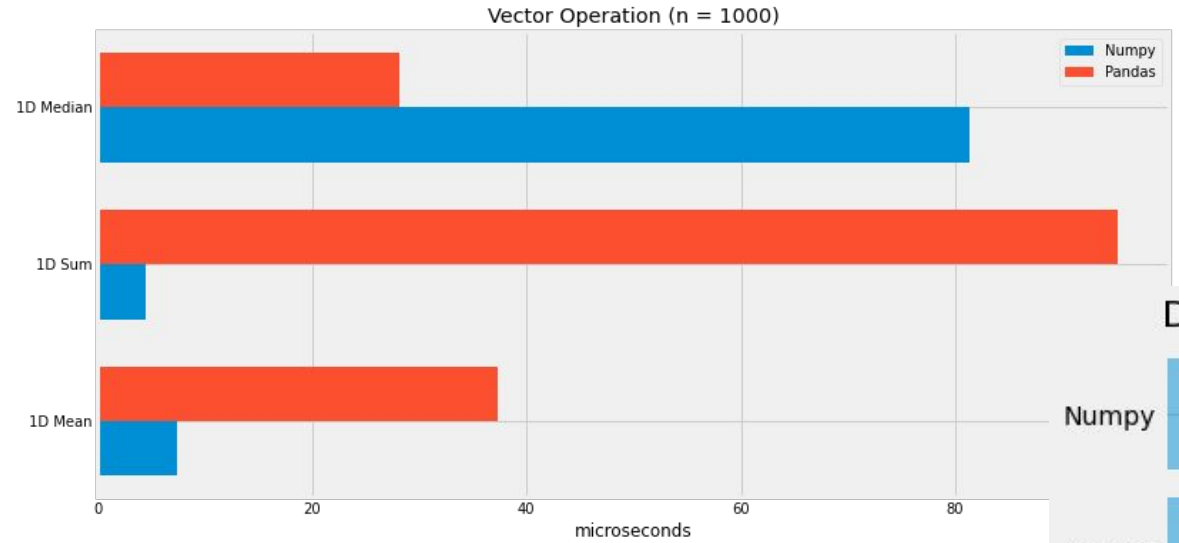


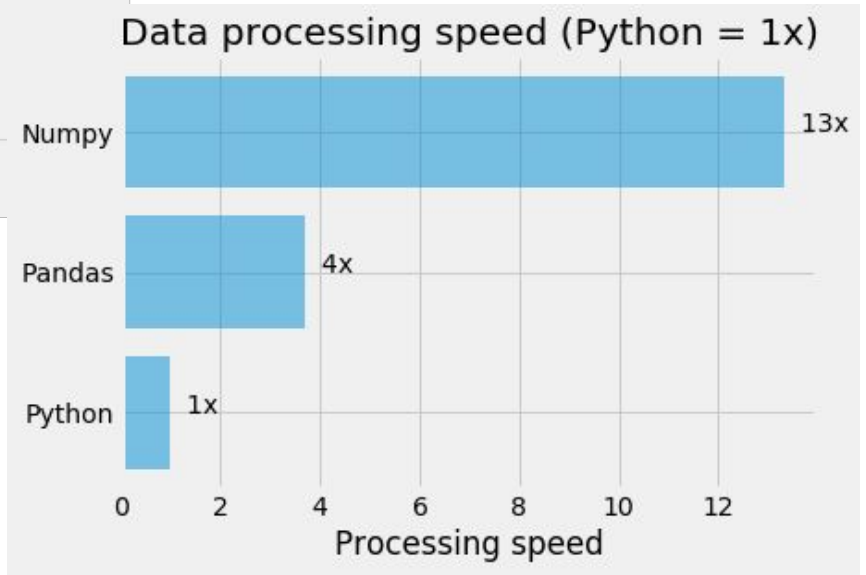
- ➔ Python não oferece vetores ou matrizes, mas oferece listas, dicionários, conjuntos de tuplas

- ➔ Em algoritmos que envolvem computação científica, manipular dados em formas como vetores e matrizes pode ser mais prático e eficiente

- ➔ NumPy foi escrito (em sua maior parte) em linguagem C, que é uma linguagem de baixo nível, o que torna a biblioteca extremamente veloz, escondendo toda sua complexidade em um módulo Python simples de utilizar

Numerical Python - NumPy





O NumPy, por esta característica, só trabalha com dados homogêneos (em sua maioria numéricos),

Numerical Python - NumPy - numpy_01.py

```
# Instalando o NumPy
pip install numpy

import numpy as np
# CRIAÇÃO DE ARRAYS
a = np.array([10, 20, 30, 40, 50])
print(a)

# Array de zeros
zeros = np.zeros(5)

# Array de uns
uns = np.ones(5)

# Sequência numérica
seq = np.arange(1, 11)
```

Numerical Python - NumPy - numpy_01.py

```
# Sequência com espaçamento definido
lin = np.linspace(0, 100, 5)

# INFORMAÇÕES DO ARRAY
print(a.ndim)       # número de dimensões
print(a.shape)      # formato
print(a.size)       # quantidade de elementos
print(a.dtype)      # tipo dos dados

# ACESSO A ELEMENTOS
print(a[0])         # primeiro elemento
print(a[-1])        # último elemento
print(a[1:4])       # fatiamento

# OPERAÇÕES MATEMÁTICAS
print(a + 10)
print(a - 5)
print(a * 2)
print(a / 2)
```

Numerical Python - NumPy - numpy_01.py

```
# OPERAÇÕES ENTRE ARRAYS
b = np.array([1, 2, 3, 4, 5])

print(a + b)
print(a * b)

# FUNÇÕES ESTATÍSTICAS
print(np.sum(a))
print(np.mean(a))
print(np.median(a))
print(np.std(a))
print(np.min(a))
print(np.max(a))

# FILTROS (MÁSCARAS BOOLEANAS)
print(a > 25)
print(a[a > 25])
```

Numerical Python - NumPy - numpy_01.py

```
# ARRAYS BIDIMENSIONAIS (MATRIZES)
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(matriz)
print(matriz.shape)
print(matriz[1, 2])     # linha 1 coluna 2

# REDIMENSIONAMENTO
v = np.arange(1, 13)
m = v.reshape(3, 4)
print(m)

# TRANSPOSIÇÃO
print(m.T)
```

Numerical Python - NumPy - numpy_01.py

```
# CONCATENAÇÃO
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
z = np.concatenate((x, y))
print(z)

# ORDENAÇÃO
dados = np.array([9, 2, 5, 1, 7])
print(np.sort(dados))

# LOCALIZAÇÃO DE ELEMENTOS
print(np.where(dados > 5))

# PRODUTO ESCALAR
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
print(np.dot(u, v))
```

Numerical Python - NumPy - numpy_01.py

```
# NÚMEROS ALEATÓRIOS
print(np.random.rand(5))
print(np.random.randint(1, 101, 10))

# CÓPIA E VISÃO
copia = a.copy()
view = a.view()

# INSERÇÃO E REMOÇÃO
novo = np.insert(a, 2, 99)
print(novo)
novo = np.delete(novo, 3)
print(novo)
```

Numerical Python - NumPy

Funcionalidade	Exemplo
Criar array	np.array()
Zeros	np.zeros()
Uns	np.ones()
Sequência	np.arange()
Intervalo contínuo	np.linspace()
Soma	np.sum()
Média	np.mean()
Desvio padrão	np.std()
Máximo/Mínimo	np.max(), np.min()
Filtrar valores	a[a > 10]
Redimensionar	reshape()
Transpor	.T
Concatenar	np.concatenate()
Ordenar	np.sort()
Localizar	np.where()
Produto escalar	np.dot()
Aleatórios	np.random.*
Inserir	np.insert()
Remover	np.delete()

UFG INF
INSTITUTO DE INFORMÁTICA

Numerical Python - NumPy - numpy_02.py

```
import numpy as np

a = np.array([1, 2, 3])
print(a.dtype)

a = np.array([1, 2, 3], dtype=np.float64)
print(a)
print(a.dtype)

b = a.astype(float)
print(b)
print(b.dtype)

a = np.array([1.7, 2.8, 3.9])
b = a.astype(int)
print(b)
```

Numerical Python - NumPy - numpy_02.py

```
a = np.array([10, 20, 30])
b = a.astype(str)
print(b)
print(b.dtype)

a = np.array(['1', '2', '3'])
b = a.astype(int)
print(b)
```

Numerical Python - NumPy - numpy_02.py

```
a = np.array([1, 2, 3])
a.astype(np.int8)
a.astype(np.int16)
a.astype(np.int32)
a.astype(np.int64)
a.astype(np.uint8)
a.astype(np.uint16)
a.astype(np.uint32)
a.astype(np.uint64)
a.astype(np.float16)
a.astype(np.float32)
a.astype(np.float64)
a.astype(np.complex64)
a.astype(np.complex128)
a.astype(np.bool_)
a.astype(np.str_)
```

Numerical Python - NumPy - numpy_02.py

```
a = np.array([1, 2, 3, 4], dtype=np.int32)
a.dtype = np.int16
print(a)
print(a.dtype)

a = np.array([1, 2, 3], dtype=np.int32)
b = a.view(np.uint32)
print(b)

a = np.array([100])
x = a.item()
print(type(x))
```

Numerical Python - NumPy - numpy_02.py

```
a = np.array([1, 2, 3])
lista = a.tolist()
print(type(lista))
print(lista)

a = np.array([1, 2, 3], dtype=np.int32)
b = a.astype(np.int32, copy=False)
print(a is b)

a = np.array([1, 2, 3])
print(a.astype(np.float32))
print(a.astype(np.float64))
print(a.astype(np.complex128))
print(a.astype(np.bool_))
```

Numerical Python - NumPy - numpy_02.py

```
dt = np.dtype([
('nome', 'U20'),
('idade', 'i4'),
('salario', 'f8')
])
dados = np.array([
('Ana', 25, 3500.0),
('João', 30, 5200.0)
], dtype=dt)
print(dados.dtype)

print(dados['nome'])
print(dados['idade'])
print(dados['salario'])
```

Numerical Python - NumPy - numpy_02.py

```
a = np.array([1, 2, 3], dtype=np.float64)
print(a.dtype)
print(a.dtype.name)
print(a.dtype.itemsize)
print(a.dtype.kind)
```

Universidade Federal de Goiás
Instituto de Informática
Prof. Dr. Ronaldo Martins da Costa

UFG
INF
INSTITUTO DE INFORMÁTICA

